DATABASE DESIGN AND MANAGEMENT SYSTEM

Project Report

Technologies: Python, MySQL, SQL

Submitted by: Avinash Pandey

Course: Master of Computer Applications (MCA)

Institute: LNCT Group of Colleges, Bhopal

Session: 2024 - 2026

Guide / Supervisor: Dr. Neelu Singh, Assistant Professor, MCA, LNCT

Date of Submission: 15 April 2026

Declaration

I, Avinash Pandey, a student of MCA at LNCT Group of Colleges, Bhopal, hereby declare that the project report titled **"Database Design and Management System"** is my own work, carried out as part of my academic curriculum. The work has not been submitted to any other institute or university for any degree or diploma.

Signature: _____ Date: _____

Acknowledgement

I thank my project guide and the faculty of the MCA department for their guidance and support, and my friends and family for their encouragement during this project.

Table of Contents

1. Abstract
2. Introduction
3. Objectives
4. Tools and Technologies
5. System Requirements
6. Database Design (ER Diagram, Normalization, Schema)
7. Data Dictionary
8. Implementation Modules
9. Advanced SQL Queries
10. Index Design and Optimization
11. Testing and Results
12. How to Run the Project
13. Conclusion
14. Future Scope
15. References
Appendix A: schema.sql
Appendix B: queries.sql
Appendix C: indexes.sql
Appendix D: db.py
Appendix E: seed.py
Appendix F: benchmark.py
Appendix G: data_dictionary.py
Appendix H: main.py

1. Abstract

This project presents the design and implementation of a relational Database Management System for a retail ordering business using MySQL and Python. The database stores customers, products, categories, orders, order items and payments in six normalized tables connected by foreign keys. The project demonstrates complex SQL (joins, subqueries, aggregations and window functions), index design and performance measurement with EXPLAIN, transaction-safe order processing, and automatic generation of a data dictionary. A menu-driven Python application ties these parts together.

2. Introduction

Databases are the foundation of almost every business application. A poorly designed database leads to duplicated data, inconsistent records and slow queries. This project applies database design principles (normalization, constraints, indexing) to a realistic retail scenario, and uses Python to automate setup, data loading, reporting and documentation.

3. Objectives

- Design a normalized (3NF) relational schema with primary keys, foreign keys and integrity constraints.
- Write complex SQL queries using joins, subqueries, aggregations and window functions.
- Design indexes and measure their effect on query execution time using EXPLAIN and timing.
- Generate a data dictionary and documentation automatically from the database metadata.
- Build a Python CLI that performs transaction-safe operations (placing orders with stock control).

4. Tools and Technologies

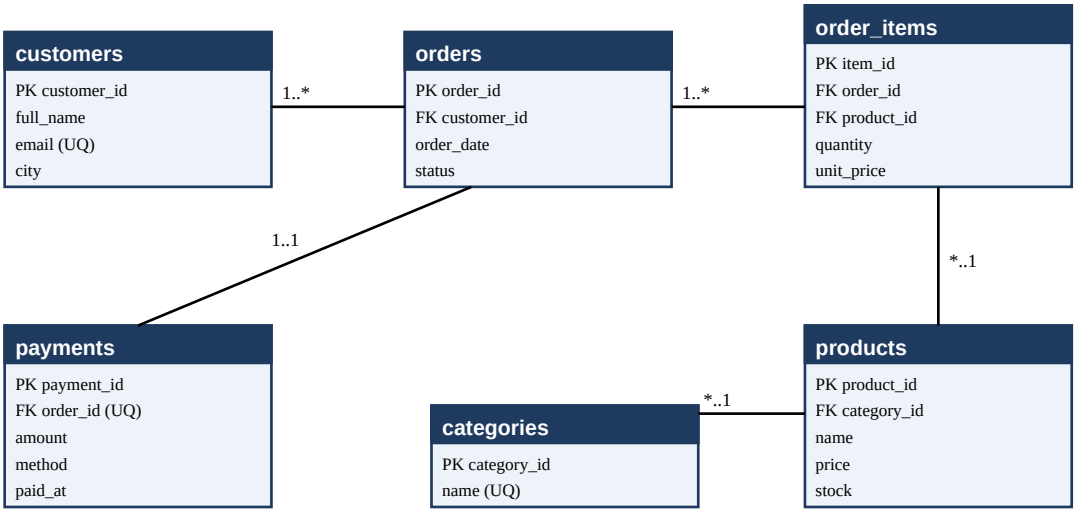
Component	Technology
Database	MySQL 8.0+ (InnoDB engine)
Language	Python 3.x
Connector	mysql-connector-python
Query tools	MySQL Workbench / mysql CLI
Editor	Visual Studio Code

5. System Requirements

- Hardware: any PC with 4 GB RAM or more.
- Software: MySQL Server 8.0 or newer (needed for window functions), Python 3.8 or newer.
- Python package: `mysql-connector-python`.

6. Database Design

6.1 Entity Relationship Diagram



Relationships: one customer places many orders; one order has many order items; each order item refers to one product; each product belongs to one category; each order has at most one payment.

6.2 Normalization

- **1NF:** all columns hold atomic values; every table has a primary key.
- **2NF:** no partial dependencies; order line data lives in *order_items*, separate from order headers.
- **3NF:** no transitive dependencies; category names are stored once in *categories* and customer details once in *customers*.
- **Design choice:** *unit_price* is copied into *order_items* so past orders keep the price at purchase time even if the product price changes later.

6.3 Integrity Constraints

- PRIMARY KEY on every table, FOREIGN KEY for all relationships.
- UNIQUE: customers.email, categories.name, payments.order_id.
- CHECK: price > 0, stock >= 0, quantity > 0.
- ENUM: orders.status and payments.method restrict values to valid options.
- ON DELETE CASCADE from orders to order_items and payments.

7. Data Dictionary

Table: customers

Column	Type	Constraints	Description
customer_id	INT	PK, AUTO_INCREMENT	Unique customer id
full_name	VARCHAR(100)	NOT NULL	Customer name
email	VARCHAR(120)	NOT NULL, UNIQUE	Email address
phone	VARCHAR(15)	NULL	Contact number
city	VARCHAR(60)	NOT NULL	City
created_at	TIMESTAMP	DEFAULT NOW	Registration time

Table: categories

Column	Type	Constraints	Description
category_id	INT	PK, AUTO_INCREMENT	Category id
name	VARCHAR(60)	NOT NULL, UNIQUE	Category name

Table: products

Column	Type	Constraints	Description
product_id	INT	PK, AUTO_INCREMENT	Product id
name	VARCHAR(120)	NOT NULL	Product name
category_id	INT	FK -> categories	Category
price	DECIMAL(10,2)	NOT NULL, CHECK > 0	Unit price
stock	INT	NOT NULL, CHECK >= 0	Units in stock

Table: orders

Column	Type	Constraints	Description
order_id	INT	PK, AUTO_INCREMENT	Order id
customer_id	INT	FK -> customers	Who ordered
order_date	DATE	NOT NULL	Order date
status	ENUM	PENDING/SHIPPED/DELIVERED/CANCELLED	Order status

Table: order_items

Column	Type	Constraints	Description
item_id	INT	PK, AUTO_INCREMENT	Line id
order_id	INT	FK -> orders (CASCADE)	Parent order
product_id	INT	FK -> products	Product ordered
quantity	INT	NOT NULL, CHECK > 0	Units
unit_price	DECIMAL(10,2)	NOT NULL	Price at purchase

Table: payments

Column	Type	Constraints	Description
payment_id	INT	PK, AUTO_INCREMENT	Payment id
order_id	INT	FK, UNIQUE (CASCADE)	Order paid
amount	DECIMAL(12,2)	NOT NULL	Amount paid
method	ENUM	UPI/CARD/COD/NETBANKING	Payment mode
paid_at	DATETIME	NOT NULL	Payment time

The file `data_dictionary.py` regenerates this dictionary automatically from MySQL's information_schema into `DATA_DICTIONARY.md`, so documentation always matches the live database.

8. Implementation Modules

File	Purpose
sql/schema.sql	Creates the database, tables, keys and constraints.
sql/queries.sql	Ten advanced analytical queries.
sql/indexes.sql	Index definitions used for optimization.
db.py	Connection helper and SQL file runner (credentials via environment variables).
seed.py	Generates about 50,000 orders of realistic sample data using executemany.
benchmark.py	Times queries before and after indexing and prints EXPLAIN access types.
data_dictionary.py	Builds DATA_DICTIONARY.md from information_schema.
main.py	Menu-driven CLI: schema setup, seeding, add customer, place order, reports, benchmark.

8.1 Transaction-safe order placement

The *place_order* function locks the product row with `SELECT ... FOR UPDATE`, checks stock, inserts the order, order item and payment, and reduces stock. If any step fails, the whole transaction is rolled back, so stock can never go negative and no partial orders are saved.

9. Advanced SQL Queries

#	Technique	Business question
1	Multi-table JOIN	Order details with customer, product and category
2	LEFT JOIN	Customers who never ordered
3	GROUP BY, SUM, COUNT	Monthly revenue
4	HAVING	Categories with revenue above 100000
5	Derived table + scalar subquery	Customers spending above the average
6	Correlated subquery	Products priced above their category average
7	RANK() OVER PARTITION BY	Product ranking within category by revenue
8	SUM() OVER (running total)	Cumulative daily revenue
9	LAG()	Days between a customer's consecutive orders
10	ROW_NUMBER()	Top 3 customers per city by spend

The full SQL text is in Appendix B.

10. Index Design and Optimization

Indexes were added where queries filter on columns that are not already covered by primary or foreign keys.

Index	Table(columns)	Reason
idx_orders_date_status	orders(order_date, status)	Range filter on date plus equality on status; the equality/range order is chosen so the date range can use the index and status is checked inside it.
idx_customers_city	customers(city)	Equality filter by city
idx_products_name	products(name)	Product lookup by name
idx_payments_paid_at	payments(paid_at)	Time-window payment reports

Method: benchmark.py removes the indexes, runs each query 20 times and records the median time, creates the indexes, runs ANALYZE TABLE, and repeats the measurement. EXPLAIN is used to compare the access type (ALL = full table scan, range/ref = index used) and estimated rows examined.

11. Testing and Results

Record your own measured values from running `python benchmark.py` in the table below. Results depend on the machine and data size, so use the numbers you obtain.

Query	Before (ms)	After (ms)	Improvement %	Access type before / after
Orders by date range and status				
Customers in a city				
Product lookup by name				
Payments in a time window				

Functional test cases

Test	Input	Expected result
Place order within stock	Valid customer, product, qty less than or equal to stock	Order, item, payment created; stock reduced
Place order over stock	qty greater than stock	Rollback with message; no rows inserted
Duplicate email	Existing email	MySQL UNIQUE violation; no insert
Invalid price	price = 0 or negative	CHECK constraint rejects row
Delete an order	DELETE FROM orders	Items and payment removed via CASCADE

12. How to Run the Project

- Install MySQL 8.0+ and Python 3.
- Run: `pip install -r requirements.txt`
- Set credentials: `set DB_USER=root` and `set DB_PASSWORD=yourpassword` (Windows) or `export DB_USER=root DB_PASSWORD=yourpassword` (Linux/Mac).
- Run: `python main.py`
- Choose option 1 to create the schema and option 2 to load sample data.
- Use options 3 to 8 for operations and reports, 9 for the index benchmark, 10 to generate the data dictionary.

13. Conclusion

The project shows the complete lifecycle of a small database system: requirement analysis, normalized design, implementation with constraints, advanced querying, performance tuning with indexes, transactional application logic and automated documentation. It strengthened my understanding of SQL, query optimization and database architecture.

14. Future Scope

- Web front end using Python Flask with login and roles.
- Stored procedures and triggers for automatic stock alerts.
- Dashboards in Power BI connected to the database.
- Table partitioning and read replicas for larger data volumes.
- Demand forecasting using machine learning on order history.

15. References

- MySQL 8.0 Reference Manual - dev.mysql.com/doc
- mysql-connector-python documentation
- IBM Cognitive Class: SQL and Relational Databases 101
- Elmasri and Navathe, Fundamentals of Database Systems
- Silberschatz, Korth and Sudarshan, Database System Concepts

Appendix A: sql/schema.sql

```
-- Database Design and Management System (Retail Orders)
-- Requires MySQL 8.0+ (window functions)

CREATE DATABASE IF NOT EXISTS retail_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE retail_db;

DROP TABLE IF EXISTS payments;
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS categories;
DROP TABLE IF EXISTS customers;

CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name   VARCHAR(100) NOT NULL,
    email       VARCHAR(120) NOT NULL UNIQUE,
    phone       VARCHAR(15),
    city        VARCHAR(60)  NOT NULL,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(60) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE products (
    product_id  INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(120) NOT NULL,
    category_id INT NOT NULL,
    price       DECIMAL(10,2) NOT NULL CHECK (price > 0),
    stock       INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
    CONSTRAINT fk_products_category
        FOREIGN KEY (category_id) REFERENCES categories(category_id)
) ENGINE=InnoDB;

CREATE TABLE orders (
    order_id    INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date  DATE NOT NULL,
    status      ENUM('PENDING','SHIPPED','DELIVERED','CANCELLED') NOT NULL DEFAULT 'PENDING',
    CONSTRAINT fk_orders_customer
        FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
) ENGINE=InnoDB;

CREATE TABLE order_items (
    item_id    INT AUTO_INCREMENT PRIMARY KEY,
    order_id   INT NOT NULL,
    product_id INT NOT NULL,
    quantity   INT NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_items_order
        FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE,
    CONSTRAINT fk_items_product
        FOREIGN KEY (product_id) REFERENCES products(product_id)
) ENGINE=InnoDB;

CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id   INT NOT NULL UNIQUE,

    amount     DECIMAL(12,2) NOT NULL,
    method     ENUM('UPI','CARD','COD','NETBANKING') NOT NULL,
    paid_at    DATETIME NOT NULL,
    CONSTRAINT fk_payments_order
        FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Appendix B: sql/queries.sql

```
-- Advanced queries: joins, subqueries, aggregations, window functions
USE retail_db;

-- 1. JOIN: order details with customer, product and category
SELECT o.order_id, c.full_name, p.name AS product, cat.name AS category,
       oi.quantity, oi.unit_price, (oi.quantity * oi.unit_price) AS line_total
FROM orders o
JOIN customers   c   ON c.customer_id = o.customer_id
JOIN order_items oi  ON oi.order_id   = o.order_id
JOIN products    p   ON p.product_id  = oi.product_id
JOIN categories  cat ON cat.category_id = p.category_id
WHERE o.status <> 'CANCELLED'
ORDER BY o.order_id
LIMIT 20;

-- 2. LEFT JOIN: customers who never placed an order
SELECT c.customer_id, c.full_name, c.city
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- 3. AGGREGATION: monthly revenue (excluding cancelled orders)
SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,
       COUNT(DISTINCT o.order_id)          AS orders,
       SUM(oi.quantity * oi.unit_price)    AS revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status <> 'CANCELLED'
GROUP BY month
ORDER BY month;

-- 4. AGGREGATION + HAVING: categories with revenue above 100000
SELECT cat.name, SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN products   p   ON p.product_id = oi.product_id
JOIN categories cat ON cat.category_id = p.category_id
GROUP BY cat.name
HAVING revenue > 100000
ORDER BY revenue DESC;

-- 5. SUBQUERY: customers whose total spend is above the average customer spend
SELECT c.customer_id, c.full_name, t.total_spent
FROM customers c
JOIN (
    SELECT o.customer_id, SUM(oi.quantity * oi.unit_price) AS total_spent
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    WHERE o.status <> 'CANCELLED'
    GROUP BY o.customer_id
) t ON t.customer_id = c.customer_id
WHERE t.total_spent > (
    SELECT AVG(x.total) FROM (
        SELECT SUM(oi.quantity * oi.unit_price) AS total
        FROM orders o
        JOIN order_items oi ON oi.order_id = o.order_id
        WHERE o.status <> 'CANCELLED'
        GROUP BY o.customer_id
    ) x
)
ORDER BY t.total_spent DESC
LIMIT 10;

-- 6. CORRELATED SUBQUERY: products priced above their category average
SELECT p.product_id, p.name, p.price
FROM products p
WHERE p.price > (SELECT AVG(p2.price) FROM products p2
                 WHERE p2.category_id = p.category_id);

-- 7. WINDOW: rank products by revenue inside each category
SELECT category, product, revenue,
       RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rank_in_category
FROM (
    SELECT cat.name AS category, p.name AS product,
           SUM(oi.quantity * oi.unit_price) AS revenue
    FROM order_items oi
    JOIN products   p   ON p.product_id = oi.product_id
    JOIN categories cat ON cat.category_id = p.category_id
    GROUP BY cat.name, p.name
```

```
) r;

-- 8. WINDOW: running total of daily revenue
SELECT order_date, daily_revenue,
       SUM(daily_revenue) OVER (ORDER BY order_date) AS running_total
FROM (
    SELECT o.order_date, SUM(oi.quantity * oi.unit_price) AS daily_revenue
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    WHERE o.status <> 'CANCELLED'
    GROUP BY o.order_date
) d
ORDER BY order_date;

-- 9. WINDOW: previous order date and gap in days per customer (LAG)
SELECT customer_id, order_id, order_date,
       LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_order_date,
       DATEDIFF(order_date,
                LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)) AS days_since_prev
FROM orders
ORDER BY customer_id, order_date
LIMIT 30;

-- 10. WINDOW: top 3 customers per city by spend (ROW_NUMBER)
SELECT * FROM (
    SELECT c.city, c.full_name, SUM(oi.quantity * oi.unit_price) AS spent,
           ROW_NUMBER() OVER (PARTITION BY c.city
                              ORDER BY SUM(oi.quantity * oi.unit_price) DESC) AS rn
    FROM customers c
    JOIN orders o       ON o.customer_id = c.customer_id
    JOIN order_items oi ON oi.order_id   = o.order_id
    WHERE o.status <> 'CANCELLED'
    GROUP BY c.city, c.customer_id, c.full_name
) t
WHERE rn <= 3;
```

Appendix C: sql/indexes.sql

```
-- Indexes added after profiling with EXPLAIN (see benchmark.py)
USE retail_db;

CREATE INDEX idx_orders_date_status ON orders (order_date, status);
CREATE INDEX idx_customers_city     ON customers (city);
CREATE INDEX idx_products_name      ON products (name);
CREATE INDEX idx_payments_paid_at   ON payments (paid_at);
```

Appendix D: db.py

```
"""Database connection helper."""
import os
import mysql.connector

CONFIG = {
    "host": os.getenv("DB_HOST", "localhost"),
    "user": os.getenv("DB_USER", "root"),
    "password": os.getenv("DB_PASSWORD", ""),
    "database": os.getenv("DB_NAME", "retail_db"),
}


def get_connection(with_db=True):
    cfg = dict(CONFIG)
    if not with_db:
        cfg.pop("database")
    return mysql.connector.connect(**cfg)


def run_sql_file(path):
    """Execute every statement in a .sql file."""
    with open(path, encoding="utf-8") as f:
        script = f.read()
    conn = get_connection(with_db=False)
    cur = conn.cursor()
    for _ in cur.execute(script, multi=True):
        pass
    conn.commit()
    cur.close()
    conn.close()
```

Appendix E: seed.py

```
"""Populate retail_db with realistic random data (50,000 orders by default)."""
import random
import sys
from datetime import date, datetime, timedelta

from db import get_connection

CITIES = ["Bhopal", "Indore", "Patna", "Delhi", "Mumbai", "Pune",
          "Lucknow", "Kolkata", "Jaipur", "Ranchi"]
CATEGORIES = ["Grocery", "Dairy", "Snacks", "Beverages", "Personal Care",
              "Household", "Stationery", "Electronics"]
FIRST = ["Aarav", "Riya", "Vivek", "Ananya", "Rohit", "Sneha", "Amit", "Pooja",
         "Karan", "Neha", "Rahul", "Isha", "Manish", "Kavya", "Suresh", "Divya"]
LAST = ["Sharma", "Verma", "Singh", "Pandey", "Gupta", "Mishra", "Yadav", "Jha"]
METHODS = ["UPI", "CARD", "COD", "NETBANKING"]
STATUSES = ["PENDING", "SHIPPED", "DELIVERED", "DELIVERED", "DELIVERED", "CANCELLED"]


def seed(n_customers=2000, n_products=200, n_orders=50000):
    random.seed(42)
    conn = get_connection()
    cur = conn.cursor()

    cur.executemany("INSERT INTO categories (name) VALUES (%s)",
                    [(c,) for c in CATEGORIES])

    products = []
    for i in range(1, n_products + 1):
        products.append((f"Product {i:03d}", random.randint(1, len(CATEGORIES)),
                         round(random.uniform(10, 2500), 2), random.randint(0, 500)))
    cur.executemany("INSERT INTO products (name, category_id, price, stock) "
                    "VALUES (%s,%s,%s,%s)", products)
    prices = {i + 1: p[2] for i, p in enumerate(products)}

    customers = []
    for i in range(1, n_customers + 1):
        name = f"{random.choice(FIRST)} {random.choice(LAST)}"
        customers.append((name, f"user{i}@example.com",
                          f"9{random.randint(100000000, 999999999)}",
                          random.choice(CITIES)))
    cur.executemany("INSERT INTO customers (full_name, email, phone, city) "
                    "VALUES (%s,%s,%s,%s)", customers)

    start = date.today() - timedelta(days=730)
    # leave ~5% of customers without orders so the LEFT JOIN query returns rows
    active = int(n_customers * 0.95)
    orders = [(random.randint(1, active),
               start + timedelta(days=random.randint(0, 730)),
               random.choice(STATUSES)) for _ in range(n_orders)]
    cur.executemany("INSERT INTO orders (customer_id, order_date, status) "
                    "VALUES (%s,%s,%s)", orders)

    items, totals = [], {}
    for oid in range(1, n_orders + 1):
        for pid in random.sample(range(1, n_products + 1), random.randint(1, 4)):
            qty = random.randint(1, 5)
            items.append((oid, pid, qty, prices[pid]))
            totals[oid] = totals.get(oid, 0) + qty * prices[pid]
    cur.executemany("INSERT INTO order_items (order_id, product_id, quantity, unit_price) "
                    "VALUES (%s,%s,%s,%s)", items)

    payments = []

    for oid, (_, odate, status) in enumerate(orders, start=1):
        if status != "CANCELLED":
            paid = datetime.combine(odate, datetime.min.time()) + timedelta(
                hours=random.randint(0, 48))
            payments.append((oid, round(totals[oid], 2), random.choice(METHODS), paid))
    cur.executemany("INSERT INTO payments (order_id, amount, method, paid_at) "
                    "VALUES (%s,%s,%s,%s)", payments)

    conn.commit()
    cur.close()
    conn.close()
    print(f"Seeded {n_customers} customers, {n_products} products, "
          f"{n_orders} orders, {len(items)} items, {len(payments)} payments.")

if __name__ == "__main__":
```

```
n = int(sys.argv[1]) if len(sys.argv) > 1 else 50000
seed(n_orders=n)
```

Appendix F: benchmark.py

```
"""Measure query time before and after adding indexes, and show EXPLAIN plans.

Run:  python benchmark.py
The improvement printed is MEASURED on your machine, not a fixed number.
"""
import time

from db import get_connection

INDEXES = {
    "idx_orders_date_status": "CREATE INDEX idx_orders_date_status ON orders (order_date, status)",
    "idx_customers_city": "CREATE INDEX idx_customers_city ON customers (city)",
    "idx_products_name": "CREATE INDEX idx_products_name ON products (name)",
    "idx_payments_paid_at": "CREATE INDEX idx_payments_paid_at ON payments (paid_at)",
}
TABLE_OF = {"idx_orders_date_status": "orders", "idx_customers_city": "customers",
            "idx_products_name": "products", "idx_payments_paid_at": "payments"}

QUERIES = {
    "Orders in a date range with status": (
        "SELECT COUNT(*) FROM orders "
        "WHERE order_date BETWEEN %s AND %s AND status = 'DELIVERED'",
        ("2026-01-01", "2026-03-31")),
    "Customers in a city": (
        "SELECT COUNT(*) FROM customers WHERE city = %s", ("Bhopal",)),
    "Product lookup by name": (
        "SELECT * FROM products WHERE name = %s", ("Product 150",)),
    "Payments in a time window": (
        "SELECT SUM(amount) FROM payments WHERE paid_at BETWEEN %s AND %s",
        ("2026-01-01", "2026-02-01")),
}


def time_query(cur, sql, params, runs=20):
    times = []
    for _ in range(runs):
        t = time.perf_counter()
        cur.execute(sql, params)
        cur.fetchall()
        times.append(time.perf_counter() - t)
    times.sort()
    return times[len(times) // 2] * 1000  # median, ms


def explain(cur, sql, params):
    cur.execute("EXPLAIN " + sql, params)
    row = cur.fetchone()
    cols = [d[0] for d in cur.description]
    d = dict(zip(cols, row))
    return d.get("type"), d.get("key"), d.get("rows")


def drop_indexes(cur):
    for name, table in TABLE_OF.items():
        cur.execute("SELECT COUNT(*) FROM information_schema.statistics "
                    "WHERE table_schema = DATABASE() AND table_name = %s AND index_name = %s",
                    (table, name))
        if cur.fetchone()[0]:
            cur.execute(f"DROP INDEX {name} ON {table}")


def main():
    conn = get_connection()
    cur = conn.cursor()
    drop_indexes(cur)

    before = {}
    for label, (sql, params) in QUERIES.items():
        before[label] = (time_query(cur, sql, params), explain(cur, sql, params))

    for ddl in INDEXES.values():
        cur.execute(ddl)
    conn.commit()
    cur.execute("ANALYZE TABLE orders, customers, products, payments")
    cur.fetchall()

    print(f"{'Query':40} {'Before ms':>10} {'After ms':>10} {'Faster by':>10}  Plan (before -> after)")
```

```
    print("-" * 110)
    for label, (sql, params) in QUERIES.items():
        after_ms = time_query(cur, sql, params)
        after_plan = explain(cur, sql, params)
        b_ms, b_plan = before[label]
        gain = (b_ms - after_ms) / b_ms * 100 if b_ms else 0
        print(f"{label:40} {b_ms:10.2f} {after_ms:10.2f} {gain:9.1f}%  "
              f"{b_plan[0]}/{b_plan[2]} rows -> {after_plan[0]}/{after_plan[2]} rows ({after_plan[1]})")

    cur.close()
    conn.close()


if __name__ == "__main__":
    main()
```

Appendix G: data_dictionary.py

```
"""Auto-generate DATA_DICTIONARY.md from information_schema."""
from db import CONFIG, get_connection

OUT = "DATA_DICTIONARY.md"

DESCRIPTIONS = {
    "customers": "People who place orders.",
    "categories": "Product groupings.",
    "products": "Items available for sale, with price and stock.",
    "orders": "One row per customer order.",
    "order_items": "Line items belonging to an order.",
    "payments": "One payment per non-cancelled order.",
}


def generate():
    conn = get_connection()
    cur = conn.cursor()
    db = CONFIG["database"]
    lines = [f"# Data Dictionary - `{db}`", ""]

    cur.execute("SELECT table_name, table_rows FROM information_schema.tables "
                "WHERE table_schema=%s ORDER BY table_name", (db,))
    for table, rows in cur.fetchall():
        lines += [f"## {table}", DESCRIPTIONS.get(table, ""),
                  f"Approx. rows: {rows}", "",
                  "| Column | Type | Nullable | Key | Default | Extra |",
                  "|---|---|---|---|---|---|"]
        cur.execute("SELECT column_name, column_type, is_nullable, column_key, "
                    "column_default, extra FROM information_schema.columns "
                    "WHERE table_schema=%s AND table_name=%s ORDER BY ordinal_position",
                    (db, table))
        for col, typ, null, key, default, extra in cur.fetchall():
            lines.append(f"| {col} | {typ} | {null} | {key or ''} | {default if default is not None else
    ''} | {extra or ''} |")

        cur.execute("SELECT column_name, referenced_table_name, referenced_column_name "
                    "FROM information_schema.key_column_usage "
                    "WHERE table_schema=%s AND table_name=%s AND referenced_table_name IS NOT NULL",
                    (db, table))
        fks = cur.fetchall()
        if fks:
            lines += ["", "**Foreign keys:** " + ", ".join(f"`{c}` -> `{t}.{rc}`" for c, t, rc in fks)]

        cur.execute("SELECT index_name, GROUP_CONCAT(column_name ORDER BY seq_in_index), non_unique "
                    "FROM information_schema.statistics WHERE table_schema=%s AND table_name=%s "
                    "GROUP BY index_name, non_unique", (db, table))
        idx = cur.fetchall()
        if idx:
            lines += ["", "**Indexes:** " + ", ".join(
                f"`{n}` ({c}{'' if nu else ', unique'})" for n, c, nu in idx)]
        lines.append("")

    with open(OUT, "w", encoding="utf-8") as f:
        f.write("\n".join(lines))
    cur.close()
    conn.close()
    print(f"Wrote {OUT}")


if __name__ == "__main__":
    generate()
```

Appendix H: main.py

```
"""Menu-driven CLI for the Database Design and Management System."""
import os

from db import get_connection, run_sql_file
import seed
import benchmark
import data_dictionary

BASE = os.path.dirname(os.path.abspath(__file__))


def show(cur, sql, params=()):
    cur.execute(sql, params)
    rows = cur.fetchall()
    cols = [d[0] for d in cur.description]
    print("\n" + " | ".join(cols))
    print("-" * 80)
    for r in rows:
        print(" | ".join(str(v) for v in r))
    print(f"({len(rows)} rows)")


def add_customer(cur, conn):
    name = input("Name: ").strip()
    email = input("Email: ").strip()
    city = input("City: ").strip()
    if not (name and email and city):
        print("All fields are required.")
        return
    cur.execute("INSERT INTO customers (full_name, email, city) VALUES (%s,%s,%s)",
                (name, email, city))
    conn.commit()
    print("Customer added, id =", cur.lastrowid)


def place_order(cur, conn):
    """Create an order atomically: check stock, insert items, reduce stock, record payment."""
    try:
        cid = int(input("Customer id: "))
        pid = int(input("Product id: "))
        qty = int(input("Quantity: "))
    except ValueError:
        print("Enter numbers only.")
        return
    try:
        cur.execute("SELECT price, stock FROM products WHERE product_id=%s FOR UPDATE", (pid,))
        row = cur.fetchone()
        if not row:
            raise ValueError("Product not found")
        price, stock = row
        if qty <= 0 or qty > stock:
            raise ValueError(f"Invalid quantity (stock available: {stock})")
        cur.execute("INSERT INTO orders (customer_id, order_date) VALUES (%s, CURDATE())", (cid,))
        oid = cur.lastrowid
        cur.execute("INSERT INTO order_items (order_id, product_id, quantity, unit_price) "
                    "VALUES (%s,%s,%s,%s)", (oid, pid, qty, price))
        cur.execute("UPDATE products SET stock = stock - %s WHERE product_id=%s", (qty, pid))
        cur.execute("INSERT INTO payments (order_id, amount, method, paid_at) "
                    "VALUES (%s,%s,'UPI',NOW())", (oid, price * qty))
        conn.commit()
        print(f"Order {oid} placed. Total = {price * qty}")
    except Exception as e:
        conn.rollback()
        print("Order failed, rolled back:", e)


def low_stock(cur):
    show(cur, "SELECT product_id, name, stock FROM products WHERE stock < 20 ORDER BY stock")


def top_products(cur):
    show(cur, "SELECT p.name, SUM(oi.quantity) AS units, SUM(oi.quantity*oi.unit_price) AS revenue "
              "FROM order_items oi JOIN products p ON p.product_id=oi.product_id "
              "GROUP BY p.product_id, p.name ORDER BY revenue DESC LIMIT 10")


def monthly_revenue(cur):
```

```
    show(cur, "SELECT DATE_FORMAT(o.order_date,'%Y-%m') AS month, "
              "SUM(oi.quantity*oi.unit_price) AS revenue FROM orders o "
              "JOIN order_items oi ON oi.order_id=o.order_id WHERE o.status<>'CANCELLED' "
              "GROUP BY month ORDER BY month DESC LIMIT 12")


def customer_orders(cur):
    try:
        cid = int(input("Customer id: "))
    except ValueError:
        print("Enter a number.")
        return
    show(cur, "SELECT o.order_id, o.order_date, o.status, SUM(oi.quantity*oi.unit_price) AS total "
              "FROM orders o JOIN order_items oi ON oi.order_id=o.order_id "
              "WHERE o.customer_id=%s GROUP BY o.order_id ORDER BY o.order_date DESC LIMIT 20", (cid,))


def run_queries_file():
    path = os.path.join(BASE, "sql", "queries.sql")
    print("Open", path, "in MySQL Workbench / mysql CLI to run all 10 advanced queries.")


MENU = """
===== Database Design & Management System =====
 1. Initialize database (schema)
 2. Seed sample data
 3. Add customer
 4. Place order (transaction)
 5. Customer order history
 6. Top 10 products by revenue
 7. Monthly revenue
 8. Low-stock products
 9. Run index benchmark
10. Generate data dictionary
11. Where are the advanced queries?
 0. Exit
"""


def main():
    while True:
        print(MENU)
        choice = input("Choose: ").strip()
        if choice == "0":
            break
        if choice == "1":
            run_sql_file(os.path.join(BASE, "sql", "schema.sql"))

            print("Schema created.")
            continue
        if choice == "2":
            seed.seed()
            continue
        if choice == "9":
            benchmark.main()
            continue
        if choice == "10":
            data_dictionary.generate()
            continue
        if choice == "11":
            run_queries_file()
            continue
        conn = get_connection()
        cur = conn.cursor()
        try:
            actions = {"3": lambda: add_customer(cur, conn),
                       "4": lambda: place_order(cur, conn),
                       "5": lambda: customer_orders(cur),
                       "6": lambda: top_products(cur),
                       "7": lambda: monthly_revenue(cur),
                       "8": lambda: low_stock(cur)}
            actions.get(choice, lambda: print("Invalid choice"))()
        finally:
            cur.close()
            conn.close()


if __name__ == "__main__":
    main()
```